Week 2 APP DEV-2

2024 Aug04 Quiz-2

1. [Source Q2; MCQ]

Consider the below JavaScript program.

```
const first = "Hello 3";

const obj1 = {
    first: "Hello 1",
    secondFunc: function () {
        console.log(this.first);
    }
}

const obj2 = {
    first: "Hello 2",
    secondFunc: function () {
        console.log(this.first);
        this.secondFunc();
    }
}

obj1.secondFunc.apply(obj2, [2, 5, 9]);
```

What will be the output of the above program, if executed?

(a) Hello 1

(b) Hello 1Hello 2

(c) Hello 2

(d) The program will result in an infinite loop

[Source ans: c]

2. [Source Q5; MCQ]

Consider the below JavaScript program.

```
new Promise((error, pass) => {
    if (5 === "5") error(5)
    else pass(8)
}).
then(d => {
    console.log("Checkpoint 4", d);
    throw new Error(20);
    return d * 5;
})
.then(d => {
    console.log("Checkpoint 2", d);
    return d;
}, d => {
    console.log("Checkpoint 5", d.message);
    return d.message * 2;
}).catch(e => {
    console.log("Checkpoint 3", e.message);
    return e.message * 2;
}).finally(d => {
    console.log("Checkpoint 1", d);
    return d * 5;
}).then(d => {
    console.log("Checkpoint 6", d);
    return d * 5;
})
```

What will be the output of the above program?

(a) Checkpoint 4 5Checkpoint 5 20 Checkpoint 1 undefined Checkpoint 6 40

(b) Checkpoint 4 5Checkpoint 3 20 Checkpoint 1 undefined Checkpoint 6 NaN

(c) Checkpoint 5 undefinedCheckpoint 1 undefined Checkpoint 6 NaN

(d) Checkpoint 5 undefinedCheckpoint 1 16 Checkpoint 6 80

(e) Checkpoint 5 undefinedCheckpoint 1 undefined Checkpoint 6 16

[Source ans: c]

3. [Source Q7; MCQ]

Consider the below JavaScript program, and predict the output, if executed.

```
async function exam () {
    let x = await new Promise((res, rej) => res(2 || 0));
    console.log("Statement 1");
    let y = await new Promise((res, rej) => res(0 && 2));
    console.log("Statement 2", "gives result as", x + y);
}
exam();
console.log("Statement 3");
```

(a) Statement 3Statement 1 Statement 2 gives result as 4

(b) Statement 3Statement 1 Statement 2 gives result as 2

(c) Statement 3Statement 1 Statement 2 gives result as 0

(d) Statement 1Statement 3 Statement 2 gives result as 4

(e) Statement 1Statement 3 Statement 2 gives result as 2

[Source ans: b]

4. [Source Q15; MCQ]

Analyze the given HTML with embedded script.

```
<body>
  <div id="status"></div>
  <div id="result"></div>
  <script>
    function updateStatus(message) {
      document.getElementById("status").textContent = message;
    }
    function updateResult(message) {
      document.getElementById("result").textContent = message;
    }
    let promise1 = new Promise((resolve, reject) => {
      setTimeout(() => {
        resolve("Data Loaded");
      }, 1000);
    });

    let promise2 = Promise.reject("Network Error");
    promise1
      .then((result) => {
        updateStatus(result);
      })
      .catch((error) => {
        updateResult(error);
      });
    promise2
      .catch((error) => {
        updateStatus(error);
        return Promise.resolve("Retry Successful");
      })
      .then((result) => {
        updateResult(result);
      });
  </script>
</body>
```

What will be the final state of HTML elements, after all the setTimeout() have finished executing.

(a)

`#status` will be "Data Loaded", and `#result` will be "Data Save Failed".

(b)

`#status` will be "Network Error", and `#result` will be "Retry Successful".

(c)

`#status` will be "Data Loaded", and `#result` will be "Retry Successful".

(d)

`#status` will be "Network Error", and `#result` will be "Data Save Failed".

[Source ans: c]

2024 Oct27 Quiz-1

5. [Source Q4; MCQ]

Consider the following JavaScript code

```
const juices = [
    { id: 1, name: "grape", rating: 8.1 },
    { id: 2, name: "apple", rating: 5.0 },
    { id: 3, name: "orange", rating: 6.9 },
    { id: 4, name: "banana", rating: 10 },
]

mapping = { "4": 1, "3": 2, "2": 3, "1": 4 }

juices.map((juice) => { return { ...juice, rank: mapping[juice.id] } })
    .sort((a, b) => a.rank - b.rank)
    .map((juice) => { console.log(juice.name) })
```

What will be the output of the code?

(a)
apple
orange
grape
banana

(b)
banana
grape
orange
apple

(c)
banana
orange
apple
grape

(d) undefined

[Source ans: c]

6. [Source Q5; MCQ]

Consider the following JavaScript code snippet

```
var name = 'Taylor'

const person = {
    name: 'Jeremy',
    talk() {
        return () => `${this.name} says Hello!`
    }
}

talkReturn = person.talk()
console.log(talkReturn())
```

What will be the output of the code?

(a) undefined says Hello!

(b) Jeremy says Hello!

(c) Taylor says Hello!

(d) ReferenceError

[Source ans: b]

7. [Source Q6; MCQ]

Consider the following code snippet

```
function createCounter() {
  let count = 0;

  return function() {
    count++;
    setTimeout(() => {
      console.log(count);
    }, 1000);
  };
}

const counter = createCounter();
counter();
counter();
counter();
```

What will be the output **after 1 second**?

(a) 1

(b)
1
1
1

(c)
1
2
3

(d)
3
3
3

[Source ans: d]

8. [Source Q7; MCQ]

Consider the below javascript program.

```
const obj = {
    a: 10,
    b() {
        return this.a;
    }
};

const c = obj.b;
const d = obj.b.bind(obj);

console.log(c(), d());
```

What will be the output of the above program?

(a) 10 10

(b) undefined 10

(c) 10 undefined

(d) undefined undefined

(e) Error

[Source ans: b]

9. [Source Q11; MCQ]

Consider the below javascript program.

```
const numbers = [10, 20, 30];
const modifiedNumbers = numbers.reduce((acc, num) => {
    if (num % 2 === 0) {
        acc.push(num * 2);
    } else {
        acc.push(num / 2);
    }
    return acc;
}, []);

const finalOutput = modifiedNumbers.filter(num => num > 20);
console.log(finalOutput);
```

What will be the output of the above program?

(a) []

(b) [10, 20, 30]

(c) [20, 40, 60]

(d) [40, 60]

(e) [40]

[Source ans: d]

10. [Source Q13; MSQ]

Which of the following statements is/are true about JavaScript's "this" keyword behavior in different contexts?

(a) In a regular function call, "this" refers to the global object in non-strict modeand "undefined" in strict mode.

(b) In an arrow function, "this" is lexically inherited from the enclosing executioncontext.

(c) Inside a method of an object, "this" always refers to the object itself,regardless of how the method is called.

(d) The value of "this" can be explicitly set using "call()", apply(), or "bind()"methods, inside a method of an object.

[Source ans: a, b, d]

11. [Source Q15; MSQ]

Which of the following statements is/are true?

(a) call() and apply() immediately invoke the function, while bind() returns a newfunction that can be called later.

(b) call() accepts arguments as a comma-separated list, whereas apply() accepts asingle array of arguments.

(c) bind() permanently sets the “this” context for a function, but the resultingfunction must still be explicitly called to execute.

(d) All three methods are used to permanently change the “this” context offunctions.

[Source ans: a, b, c]

2024 Dec01 Quiz-2

12. [Source Q2; MCQ]

Consider the below JavaScript program.

```
function main() {
    var count = 0;
    function increment(num = count++) {
        count += num;
    }
    increment();
    increment(2);
    increment();
    console.log(count);
}
main();
```

What will be the output of the above program?

(a) 3

(b) 4

(c) 7

(d) 8

(e) 6

[Source ans: c]

13. [Source Q4; MCQ]

Consider the below JavaScript program.

```
async function asyncFunc() {
  try {
    const result1 = await Promise.resolve('Start');
    console.log(result1);

    const result2 = await new Promise((resolve, reject) => {
      setTimeout(() => reject('Error'), 100);
    });

    console.log(result2);
  } catch (error) {
    console.log('Caught:', error);
  }

  return 'End';
}

asyncFunc().then(result => console.log(result));
```

What will be the output of the program?

(a) Start, Caught: Error, End

(b) Start, End, Caught: Error

(c) Caught: Error, Start, End

(d) Start, Error, End

[Source ans: a]

14. [Source Q6; MCQ]

Consider the JavaScript code snippet below.

```
const movies = [
    { title: 'Inception', rating: 8.8 },
    { title: 'Interstellar', rating: 8.6 },
    { title: 'Dunkirk', rating: 7.9 }
];

const summaries = movies.map(({ title, rating }) => `${title} has a rating of
${rating}`);

summaries.forEach((summary, index) => {
    const delay = 1000 - (index * 200);

    setTimeout(() => {
        console.log(summary);
    }, delay);
});
```

What will be logged to the console?

(a)

Inception has a rating of 8.8
Interstellar has a rating of 8.6
Dunkirk has a rating of 7.9

(b)

Inception has a rating of 8.8
Dunkirk has a rating of 7.9
Interstellar has a rating of 8.6

(c)

Dunkirk has a rating of 7.9
Interstellar has a rating of 8.6
Inception has a rating of 8.8

(d)

Interstellar has a rating of 8.6
Inception has a rating of 8.8
Dunkirk has a rating of 7.9

[Source ans: c]

15. [Source Q7; MCQ]

Consider the below JavaScript program.

```
let a = 10;
let obj = {
    a: 20,
    fn: function() {
        let a = 30;
        return () => {
            let a = 40;
            return this.a;
        };
    }
};

console.log(obj.fn()());
```

What will be the output of the above program?

(a) 10

(b) 20

(c) 30

(d) 40

[Source ans: b]

16. [Source Q8; MCQ]

Consider the JavaScript code below.

```
console.log('Start');

Promise.resolve().then(() => {
    console.log('Promise');
});

setTimeout(() => {
  console.log('Timeout');
}, 0);

console.log('End');
```

What will be the order of logs in the console?

(a) Start, Timeout, End, Promise

(b) Start, Promise, End, Timeout

(c) Start, End, Promise, Timeout

(d) Start, Promise, Timeout, End

[Source ans: c]

17. [Source Q10; MCQ]

Consider the below JavaScript program.

```
const p1 = new Promise((resolve, reject) => {
  setTimeout(() => resolve('A'), 100);
});

const p2 = new Promise((resolve, reject) => {
  setTimeout(() => reject('B'), 50);
});

const p3 = new Promise((resolve, reject) => {
  setTimeout(() => resolve('C'), 150);
});

Promise.any([p1, p2, p3])
  .then(value => console.log(value))
  .catch(error => console.log(error));
```

What will be the output of the above program?

(a) A

(b) B

(c) C

(d) The catch block will execute

[Source ans: a]

2024 Dec22 ET

18. [Source Q2; MCQ]

Which of the following statements is true about closures in JavaScript?

(a) A closure can access variables from its outer function even after the outerfunction has returned.

(b) A closure can be created when a function is defined inside another function.

(c) Closures are useful for data encapsulation and controlling access to privatedata.

(d) All of these.

[Source ans: d]

19. [Source Q3; MCQ]

Consider the below JavaScript program.

```
console.log("Start");

setTimeout(function() {
  console.log("Inside Timeout");
}, 0);

Promise.resolve().then(function() {
  console.log("Inside Promise");
});

console.log("End");
```

What will be the output of the above program?

(a) StartInside Timeout Inside Promise End

(b) StartInside Promise Inside Timeout End

(c) StartEnd Inside Timeout Inside Promise

(d) StartEnd Inside Promise Inside Timeout

[Source ans: d]

20. [Source Q4; MCQ]

Consider the below JavaScript program.

```
function Person(name) {
  this.name = name;
}

Person.prototype.greet = function() {
  console.log("Hello, " + this.name);
};

const john = new Person("John");
john.greet();

delete john.greet;
john.greet();
```

What will be the output of the above program?

(a) Hello, JohnTypeError: john.greet is not a function

(b) Hello, JohnHello, undefined

(c) TypeError: john.greet is not a functionTypeError: john.greet is not a function

(d) Hello, JohnHello, John

[Source ans: d]

21. [Source Q13; MCQ]

What will be the output on the console?

Consider the following JavaScript code snippet.

```
Promise.resolve(1)
    .then(x => x + 1)
    .then(x => Promise.resolve(x + 1))
    .then(x => { throw 'error' })
    .catch(e => e + 4)
    .then(x => console.log(x))
```

(a) error4

(b) 7

(c) error

(d) undefined

[Source ans: a]

22. [Source Q14; MCQ]

consider the following javascript code snippet.

```
async function test() {
    console.log("Start");
    const val = await Promise.resolve(5);
    console.log(val);
    return "End";
}

console.log("Begin");
test().then(data => console.log(data));
console.log("Finish");
```

What will be the output sequence?

(a) Begin, Start, 5, End, Finish

(b) Begin, Finish, Start, 5, End

(c) Begin, Finish, Start, End, 5

(d) Begin, Start, Finish, 5, End

[Source ans: d]

23. [Source Q17; MCQ]

Which of the following statements is false when using the “async” and “await” keywords in JavaScript?

(a) The “await” keyword can only be used inside an async function.

(b) The “await” keyword pauses the execution of the surrounding async functionuntil the promise resolves.

(c) The async functions always return a promise, even if the return value is not apromise.

(d) The async functions run synchronously, but their await statements executeasynchronously.

[Source ans: d]

24. [Source Q18; MCQ]

Consider the below JavaScript program.

```
const obj = {
  name: "Abhi",
  greet: function() {
      console.log(this.name);
  }
};

const greet = obj.greet;
greet();
```

What will be the output of the above program?

(a) Abhi

(b) undefined

(c) null

(d) Reference Error

(e) Garbage Value

[Source ans: b]

25. [Source Q19; MCQ]

Consider the below JavaScript program.

```
const person = {
  firstName: "John",
  lastName: "Doe",
  fullName: function() {
      return this.firstName + " " + this.lastName;
  }
};

const newPerson = person.fullName.bind({ firstName: "Jane", lastName:
"Smith" });
console.log(newPerson());
```

What will be the output of the above program?

(a) John Doe

(b) Jane Smith

(c) John Smith

(d) Jane Doe

(e) undefined

(f) null

(g) Reference Error

[Source ans: b]

26. [Source Q21; MCQ]

Consider the following JavaScript code.

```
function add(x) {
    return function (y) {
        return x + y;
    };
}

const addFive = add(5);
console.log(addFive(10));
```

What will be the output of the code snippet?

(a) 5

(b) 10

(c) 15

(d) 20

[Source ans: c]

27. [Source Q30; MCQ]

Consider the following Script embedded in an HTML document.

```
var var1 = 56;

const exObj = {
    var2: 24,
    var1: 15,
    inObj : {
      var1: 75,
      inObjFunc: ()=>{
        return "Value is " + this.var1;
      }
    },
    exObjFunc: function(){
        let var2 = 14;
        return "Value is " + this.var2;
    }
  }

let x = exObj.inObj
console.log(x.inObjFunc())
console.log(exObj.exObjFunc())
```

What will be the output on console, if the HTML document is rendered using a browser?

(a)

```
Value is 75
Value is 24
```

(b)

```
Value is 56
Value is 24
```

(c)

```
Value is 15
Value is 14
```

(d)

```
Value is 56
Value is 14
```

[Source ans: b]

28. [Source Q31; MSQ]

Consider the following Script embedded in an HTML document.

```
let myBox = {
        tool: "Wrench",
        cutter: "Machine Cutter",
        get tools(){
            return `tool: ${this.tool}, cutter: ${this.cutter}`
        },
        set tools(q){
            let box = q.split(' ');
            this.tool = box[0];
            this.cutter = box[1];
        }
    }
```

Which of the following statements will throw an error on the console when the HTML document is rendered using a browser?

(a)

```
console.log(myBox.tools)
```

(b)

```
console.log(myBox.tools())
```

(c)

```
console.log(myBox.tools('ScrewDriver Grinder'))
```

(d)

```
console.log(myBox.tools = 'ScrewDriver Grinder')
```

[Source ans: b, c]

2025 Feb23 Quiz-1

29. [Source Q6; MCQ]

Consider the below JavaScript program.

```
function outer() {
  let count = 0;
  return function inner() {
    return ++count;
  };
}

const counter1 = outer();
const counter2 = outer();

console.log(counter1());
console.log(counter1());
console.log(counter2());
```

What will be the output of the above program?

(a) 12 3

(b) 11 1

(c) 12 1

(d) 01 0

(e) 01 1

[Source ans: c]

30. [Source Q7; MCQ]

Consider the 4 JavaScript code snippets given below.

Code Snippet 1:

```
let x = 10;
const y;
```

Code Snippet 2:

```
let { a: b } = undefined;
```

Code Snippet 3:

```
const arr = [1, 2];
arr = [3, 4];
```

Code Snippet 4:

```
const a = 10, b = 20;
(() => { return a + b; })();
```

Which of the above JavaScript code snippet(s) will throw an error?

(a) 1 and 2

(b) 1, 2 and 3

(c) 2, 3 and 4

(d) All 4

[Source ans: b]

31. [Source Q9; MCQ]

Consider the following JavaScript code snippet shown below.

```
let length = 5;

function countLen(item){
    console.log(this.length);
}

const data = [countLen, "Apple", length]

data[0]('Script')
```

What will be output on browser's console for the above JavaScript code?

(a) 6

(b) 5

(c) 4

(d) 3

[Source ans: d]

32. [Source Q10; MCQ]

Consider the following JavaScript code snippet shown below.

```
var value = 50;

const mainObj = {
    value: 42,
    getValue: function() {
      return this.value;
    }
  };

const nextObj = {
value: 100
};

const getValue1 = mainObj.getValue.bind()(nextObj);
const getValue2 = mainObj.getValue.bind(nextObj)();

console.log(getValue1);
console.log(getValue2);
```

What will be output on browser's console for the above JavaScript code?

(a)
```
50
100
```

(b)
```
42
50
```

(c)
```
100
100
```

(d)
```
undefined
100
```

(e)
```
undefined
50
```

[Source ans: d]

33. [Source Q11; MCQ]

Consider the following JavaScript code

```
let Tree = {
    name: "Oak",
    size: 5,
    description: function (size) {
        return `A ${this.size > 10 ? 'large' : 'medium'} ${this.name} tree.`;
    }
}

const willow = {
    name: 'Willow'
};

const returnedString = Tree.description.call(willow, 15);

console.log(returnedString)
```

What will be the output of the code?

(a) A large Oak tree

(b) A medium Oak tree

(c) A large Willow tree

(d) A medium Willow tree

[Source ans: d]

34. [Source Q17; MSQ]

Consider the following JavaScript Code.

```
class Person {
    constructor(name) {
        this.name = name;
    }

    greet() {
        console.log(`Hello, my name is ${this.name}`);
    }

    static identify() {
        console.log("I am a Person class");
    }
}

const john = new Person("John");
```

Which of the following expression(s) will cause an error?

(a) `john.greet();`

(b) `Person.greet();`

(c) `john.identify();`

(d) `Person.identify();`

(e) `console.log(john.name);`

[Source ans: b, c]

2025 Mar16 Quiz-2

35. [Source Q6; MCQ]

Consider the below javascript program.

```
let count = 0;

async function increment() {
    const incrementCount = () => {
        return new Promise(resolve => {
            setTimeout(() => {
                count++;
                resolve(count);
            }, 100);
        });
    };

    const results = await Promise.all([incrementCount(), incrementCount(),
incrementCount()]);
    console.log(results);
}

increment();
console.log("Start");
```

What will be the output of the above program?

(a) [1, 1, 1]
Start

(b) Start
[1, 1, 1]

(c) [1, 2, 3]
Start

(d) Start
[1, 2, 3]

[Source ans: d]

36. [Source Q7; MCQ]

Consider the below javascript program.

```
const obj = { a: 1, b: 2 };
const { a, ...rest } = obj;
console.log(a, rest);
```

What will be the output of the above program?

(a) Error

(b) { a: 1, b: 2 } undefined

(c) { a: 1, b: 2 } { a: 1, b: 2 }

(d) 1 undefined

(e) 1 { b: 2 }

[Source ans: e]

37. [Source Q11; MCQ]

Consider the following javascript code.

```
function translateToEmoji(msg, dictionary, cb) {
    const translated = msg
        .split(' ')
        .map(word => dictionary[word] || word);
    cb(translated.join(' '));
}

translateToEmoji('i love code',
    { heart: '❤️', code: '💻' },
    result => console.log(result));
```

What's logged?

(a) "i love code"

(b) "i ❤️ 💻"

(c) "undefined undefined undefined"

(d) "i love 💻"

[Source ans: d]

38. [Source Q14; MCQ]

Consider the below javascript program.

```
const promiseChain = new Promise((resolve, reject) => {
        console.log('Step 1');
        resolve(1);
    })
    .then((value) => {
        console.log('Step 2:', value);
        return value + 1;
    })
    .then((value) => {
        console.log('Step 3:', value);
        throw new Error('Error in Step 3');
    })
    .then((value) => {
        console.log('Step 4:', value);
        return value * 2;
    })
    .catch((err) => {
        console.log('Caught:', err.message);
        return 5;
    })
    .then((value) => {
        console.log('Step 5:', value);
        return value + 1;
    });

console.log('End of Code');
```

What will be the output of the above program?

(a) Step 1Step 2: 1 Step 3: 2 Caught: Error in Step 3 End of Code

(b) Step 1Step 2: 1 Caught: Error in Step 3Step 5: 5 End of Code

(c) Step 1Step 2: 1 Step 3: 2 End of Code

(d) Step 1End of Code Step 2: 1 Step 3: 2 Caught: Error in Step 3 Step 5: 5

[Source ans: d]

2025 Apr13 ET

39. [Source Q2; MCQ]

Which of the following statements is true regarding the ”this” keyword in JavaScript?

(a) It always refers to the global object.

(b) Arrow functions have their own ”this” context.

(c) ”this” in an event listener always refers to the window object.

(d) None of these

[Source ans: d]

40. [Source Q4; MCQ]

Consider the below JavaScript program.

```
const promise = new Promise((resolve, reject) => {
  resolve('Success!');
  reject('Failed!');
});
promise.then(console.log).catch(console.log);
```

What will be the output of the above program?

(a) SuccessFailed

(b) Success

(c) Failed

(d) Error

[Source ans: b]

41. [Source Q8; MCQ]

Consider the below JavaScript program.

```
let arr = [1, 2, 3];
let sum = arr.reduce((acc, val, idx, array) => acc + array[idx + 1], 0);
console.log(sum);
```

What will be the output of the above program?

(a) 6

(b) NaN

(c) undefined

(d) 0

(e) NULL

[Source ans: b]

42. [Source Q9; MCQ]

Consider the below JavaScript program.

```
const obj = {
  num: 5,
  multiply: function () {
    return this.num * 2;
  },
};
const multiply = obj.multiply;
console.log(multiply());
```

What will be the output of the above program?

(a) 10

(b) undefined

(c) NaN

(d) NULL

(e) Error

[Source ans: c]

43. [Source Q14; MCQ]

Consider the following JavaScript program, and predict the output, if executed.

```
let propA = 10;

const obj1 = {
    propA : 20,
    propB : function () {
        console.log(propA, this.propA)
    }
}

const obj2 = {
    propA : 30,
    propB : function () {

        let func = () => console.log(propA, this.propA)

        func()
    }
}

obj2.propB.call(obj1);
```

(a) 10 30

(b) 10 undefined

(c) 10 20

(d) The program will raise an error

[Source ans: c]

44. [Source Q17; MCQ]

Consider the following JavaScript code snippet. If the code is run on the browser, what will be the output on the console?

```
console.log("variable1")
setTimeout(() => {
    console.log("variable2")
    setTimeout(() => {
        console.log("variable3")
    }, 500);
}, 2000)
console.log("variable4")
```

(a)

variable1
variable2
variable3
variable4

(b)

variable1
variable4
variable3
variable2

(c)

variable1
variable4
variable2
variable3

(d)

variable1
variable3
variable4
variable2

[Source ans: c]

45. [Source Q21; MCQ]

What will be logged to the console when the following JavaScript code runs?

```
const alpha = [3, 5, 7, 9, 11, 13, 15, 17];
const beta = alpha.map(x => x ** 3);
const gamma = beta.filter(y => (y % 7 === 6) || (y % 5 === 4));
const delta = gamma.slice(0, 3);
console.log(delta);
```

(a) [27, 125, 343, 729, 1331]

(b) [125, 343, 729]

(c) [27, 125, 343]

(d) None of these

[Source ans: b]

46. [Source Q28; MSQ]

Which of the following statements are true regarding event loop in JavaScript?

(a) The event loop runs on a separate thread from the main JavaScript execution.

(b) The event loop allows JavaScript to perform non-blocking operations despitebeing single-threaded.

(c) Callbacks from setTimeout are placed in the task queue and executed after thecall stack is empty.

(d) All of these

[Source ans: b, c]

2025 Jul13 Quiz-1

47. [Source Q4; MCQ]

Consider the following code.

```
const arr = [1, 2, 3];
const result = arr.map(function(item, index) {
  return this[index] * item;
}, [10, 20, 30]);
console.log(result);
```

What will be the output?

(a) [1, 2, 3]

(b) [10, 40, 90]

(c) [10, 20, 30]

(d) [NaN, NaN, NaN]

[Source ans: b]

48. [Source Q6; MCQ]

What will be the output of this code?

```
function createFunctions() {
  var funcs = [];
  for (var i = 0; i < 3; i++) {
    funcs.push(function() {
      return i;
    });
  }
  return funcs;
}

const functions = createFunctions();
console.log(functions[0](), functions[1](), functions[2]());
```

(a) 0, 1, 2

(b) 1, 2, 3

(c) undefined, undefined, undefined

(d) 3, 3, 3

[Source ans: d]

49. [Source Q8; MCQ]

Consider the following JavaScript code:

```
const products = [
    { id: 101, name: "laptop", price: 800 },
    { id: 102, name: "mouse", price: 25 },
    { id: 103, name: "keyboard", price: 60 },
    { id: 104, name: "monitor", price: 300 }
];

const priorities = { "102": 1, "104": 2, "101": 3, "103": 4 };

products
    .filter((product) => product.price > 50)
    .map((product) => {
        return { ...product, priority: priorities[product.id] }
    })
    .sort((a, b) => a.priority - b.priority)
    .forEach((product) => {
        console.log(product.name);
    });
```

What will be the output of the code?

(a)

```
laptop
keyboard
monitor
```

(b)

```
monitor
laptop
keyboard
```

(c)

```
mouse
monitor
laptop
keyboard
```

(d)

```
laptop
mouse
keyboard
monitor
```

(e) The code will throw an error

[Source ans: b]

50. [Source Q9; MCQ]

Consider the following JavaScript code to answer the below question:

```
const grandParent = {
  username: 'GrandParent',
  getName: function () {
    return this.username;
  },
  parent: {
    username: 'Parent',
    getName: function () {
      return this.username;
    },
    child: {
      username: 'Child',
      getNameArrow: () => {
        return this.username;
      },
      getNameFunc: function () {
        return this.username;
      }
    }
  }
};

console.log("A: " + grandParent.getName());
console.log("B: " + grandParent.parent.getName());
console.log("C: " + grandParent.parent.child.getNameFunc());
console.log("D: " + grandParent.parent.child.getNameArrow());
```

What will be the result of all the console.log after execution of the above code?

(a)

```
A: GrandParent
B: Parent
C: Child
D: Reference Error
```

(b)

```
A: GrandParent
B: Parent
C: undefined
D: Child
```

(c)

```
A: GrandParent
B: GrandParent
C: Parent
D: GrandParent
```

(d)

```
A: GrandParent
B: Parent
C: Child
D: undefined
```

[Source ans: d]

51. [Source Q11; MCQ]

Consider the following JavaScript program:

```
function createMultiplier(base) {
    let multiplier = base;

    function updateMultiplier(newValue) {
        multiplier = newValue;
        return multiplier;
    }

    function multiply(num) {
        return num * multiplier;
    }

    return {
        update: updateMultiplier,
        calculate: multiply,
        getMultiplier: function() { return multiplier; }
    };
}

const math1 = createMultiplier(3);
const math2 = createMultiplier(5);

console.log(math1.calculate(4));
console.log(math2.calculate(2));
console.log(math1.update(7));
console.log(math1.calculate(4));
console.log(math2.getMultiplier());
console.log(math1.getMultiplier());
```

What will be the output of the above program?

(a) 12, 10, 7, 12, 5, 7

(b) 12, 10, 7, 28, 5, 7

(c) 12, 10, 3, 28, 5, 3

(d) 15, 10, 7, 28, 7, 7

(e) 12, 10, 7, 21, 5, 7

[Source ans: b]

52. [Source Q12; MCQ]

Consider the following JavaScript code snippet:

```
var globalScore = 25;

const player1 = {
    score: 150,
    getScore: function() {
        return this.score;
    },
    displayScore: function(bonus) {
        return this.score + (bonus || 0);
    }
};

const player2 = { score: 200 };
const player3 = { score: 75 };

const method1 = player1.getScore;
const method2 = player1.getScore.bind(player2);
const method3 = player1.displayScore.bind(player3);

console.log(method1());
console.log(method2());
console.log(method3(50));
console.log(method3.call(player1, 25));
```

What will be the output of the above JavaScript code?

(a) undefined, 200, 125, 175

(b) 25, 200, 125, 175

(c) 150, 200, 125, 100

(d) 25, 200, 125, 100

(e) undefined, 200, 125, 100

[Source ans: e]

53. [Source Q13; MSQ]

Consider the following JavaScript code to answer the below question:

```
function normalFunc() {
  console.log(this.constructor.name);
}

const arrowFunc = () => {
  console.log(this.constructor.name);
};

const objj = {
  name: 'Test',
  normalMethod: normalFunc,
  arrowMethod: arrowFunc,
  outer: function () {
    return () => {
      console.log(this.name);
    };
  }
};

const inner = objj.outer();

objj.normalMethod();
objj.arrowMethod();
inner();
new arrowFunc();
```

Which of the following statements is/are TRUE?

(a) `obj.normalMethod()` prints `"Object"`

(b) `obj.arrowMethod()` prints `"Window"` or throws an error in strict mode

(c) `inner()` prints `"Test"`

(d) `new arrowFunc()` throws an error because arrow functions cannot be used as constructors

[Source ans: a, b, c, d]

2025 Aug03 Quiz-2

54. [Source Q9; MCQ]

Consider the following JavaScript program:

```
const promise1 = new Promise((resolve, reject) => {
    setTimeout(() => {
        resolve('The cat sleeps');
    }, 2000);
});

promise1
    .then((result) => {
        console.log(result);
        return new Promise((resolve, reject) => {
            setTimeout(() => {
                resolve('Rain falls gently');
            }, 1000);
        });
    })
    .then((result) => {
        console.log(result);
        return new Promise((resolve, reject) => {
            setTimeout(() => {
                resolve('Books gather dust');
            }, 1500);
        });
    })
    .then((result) => {
        console.log(result);
        throw new Error('Coffee grows cold');
    })
    .then((result) => {
        console.log('Stars shine bright');
    })
    .catch((error) => {
        console.log("Caught:", error.message);
    });
```

What will be the output of the above program when executed?

(a)

The cat sleeps
Rain falls gently
Books gather dust
Stars shine bright

(b)

The cat sleeps
Rain falls gently
Books gather dust
Caught: Coffee grows cold

(c)

The cat sleeps
Rain falls gently
Caught: Coffee grows cold

(d)

The program will crash without showing any output

(e)

The cat sleeps
Rain falls gently
Books gather dust
Caught: Coffee grows cold
Stars shine bright

[Source ans: b]

2025 Aug31 ET

55. [Source Q12; MCQ]

What will be the order of console output when the following JavaScript code is executed?

```
async function test() {
  console.log("P");
  await Promise.resolve();
  console.log("Q");
}
console.log("R");
test();
console.log("S");
```

(a) R P Q S

(b) P R S Q

(c) R S P Q

(d) R P S Q

[Source ans: d]

56. [Source Q13; MCQ]

Consider the following JavaScript program:

```
const user = {
    name: 'Alice',
    age: 25,
    city: 'Boston',
    country: 'USA'
};

const { name, city, ...otherInfo } = user;
const newUser = { name, location: city, ...otherInfo };

console.log(name);
console.log(otherInfo);
console.log(newUser);
```

What will be the output of the above program?

(a)

```
Alice
{ city: 'Boston', age: 25, country: 'USA' }
{ name: 'Alice', location: 'Boston', city: 'Boston', age: 25,
country: 'USA' }
```

(b)

```
Alice
{ age: 25, country: 'USA' }
{ name: 'Alice', location: 'Boston', city: 'Boston', age: 25,
country: 'USA' }
```

(c)

```
Alice
{ age: 25, country: 'USA' }
{ name: 'Alice', location: 'Boston', age: 25, country: 'USA' }
```

(d)

```
undefined
{ name: 'Alice', city: 'Boston', age: 25, country: 'USA' }
{ name: 'Alice', location: 'Boston' }
```

[Source ans: c]

57. [Source Q14; MCQ]

Consider the following javascript program.

```
new Promise((resolve, reject) => {
    const score = 0.45;
    if (score >= 0.5) {
        resolve(score);
    } else {
        reject(new Error("Score too low"));
    }
})
.then(data => {
    console.log("Stage A:", data);
    return data * 10;
})
.catch(error => {
    console.log("Stage B:", error.message);
    return 3;
})
.then(data => {
    console.log("Stage C:", data);
    if (data < 5) {
        throw new Error("Value below threshold");
    }
    return data + 2;
})
.then(data => {
    console.log("Stage D:", data);
    return data / 2;
})
.catch(error => {
    console.log("Stage E:", error.message);
    if (error.message === "Value below threshold") {
        return "Recovered";
    }
    throw error;
})
.then(data => {
    console.log("Stage F:", data);
    if (data === "Recovered") {
        return 10;
    }
    return data * 4;
})
.finally(() => {
    console.log("Stage G: Cleanup completed");
});
```

Consider the following javascript program.

What will be the output of the above program on the browser's console?

(a)

```
Stage A: 0.45
Stage C: 4.5
Stage D: 6.5
Stage F: 26
Stage G: Cleanup completed
```

(b)

```
Stage B: Score too low
Stage C: 3
Stage E: Value below threshold
Stage F: Recovered
Stage G: Cleanup completed
```

(c)

```
Stage A: 0.45
Stage C: 4.5
Stage E: Value below threshold
Stage F: Recovered
Stage G: Cleanup completed
```

(d)

```
Stage B: Score too low
Stage C: 3
Stage D: 5
Stage F: 20
Stage G: Cleanup completed
```

[Source ans: b]

58. [Source Q25; MCQ]

Consider the following JavaScript code:

```
const globalVar = 100;
var value = 200;

const calculator = {
    value: 50,

    compute: function() {
        const innerObj = {
            value: 25,

            regularMethod: function() {
                return this.value;
            },

            arrowMethod: () => {
                return this.value;
            },

            mixedMethod: function() {
                const arrowInside = () => this.value;
                return arrowInside();
            }
        };
        return {
            regular: innerObj.regularMethod(),
            arrow: innerObj.arrowMethod(),
            mixed: innerObj.mixedMethod()
        };
    }
};

const result = calculator.compute();
console.log(result.regular);
console.log(result.arrow);
console.log(result.mixed);
```

Assuming this code runs in a browser environment, what will be the output?

(a)

```
25
200
50
```

(b)

```
25
50
25
```

(c)

```
50
200
50
```

(d)

```
25
200
25
```

[Source ans: b]

59. [Source Q29; MCQ]

What will this JavaScript code output?

```
function createCounter() {
    let count = 0;
    return function() {
        count++;
        return count;
    };
}

const counter1 = createCounter();
const counter2 = createCounter();

console.log(counter1());
console.log(counter1());
console.log(counter2());
```

(a)



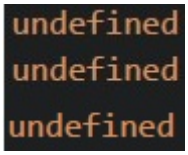
(b)



(c)



(d)



[Source ans: b]

Shared context (source Q34)

Prashant and Nikita are creating a quiz question paper collaboratively. They are each working on different sections - and complete their tasks in parallel. Meanwhile, Mayur is enjoying a single-player game that finishes when the game loop ends.

To compare productivity:

• If Prashant and Nikita (together) finish faster than Mayur, the winner is "Team". • If Mayur finishes faster, he wins. • If both take the same time, Mayur wins by default.

You are simulating this scenario in the browser.Prashant and Nikita are represented using parallel Promises. Mayur is represented using a single Promise. The frontend must calculate who finishes first using asynchronous logic, and display the result.

app.js

```
function simulateTasks() {
  const prashant = new Promise(resolve => setTimeout(() =>
resolve("Prashant"), 3000));
  const nikita = new Promise(resolve => setTimeout(() =>
resolve("Nikita"), 2000));
  const mayur = new Promise(resolve => setTimeout(() =>
resolve("Mayur"), 4000));

  const team = Promise.all([prashant, nikita]).then(() =>
"Team");

  Promise.race([team, mayur]).then(winner => {
    console.log("Winner is:", winner);
  });
}
```

Prashant and Nikita are creating a quiz question paper collaboratively. They are each working on different sections - and complete their tasks in parallel. Meanwhile, Mayur is enjoying a single-player game that finishes when the game loop ends.

To compare productivity:

• If Prashant and Nikita (together) finish faster than Mayur, the winner is "Team". • If Mayur finishes faster, he wins. • If both take the same time, Mayur wins by default.

You are simulating this scenario in the browser.Prashant and Nikita are represented using parallel Promises. Mayur is represented using a single Promise. The frontend must calculate who finishes first using asynchronous logic, and display the result.

Based on the above data, answer the given subquestions.

60. [Source Q35; MCQ]

If Prashant and Nikita finish together in 3 seconds (since Promise.all waits for both), and Mayur finishes in 4 seconds, who wins?

(a) Mayur

(b) Team

(c) Nikita

(d) Race condition prevents output

[Source ans: b]

61. [Source Q36; MCQ]

The Team takes 5 seconds in total. Mayur takes 2 seconds. According to the logic of app2.js, who is the winner?

app2.js

```
async function playGame() {
  const start = Date.now();

  const prashantPromise = new Promise(resolve =>
setTimeout(resolve, 3000));
  const nikitaPromise = new Promise(resolve =>
setTimeout(resolve, 2000));
  const mayurPromise = new Promise(resolve =>
setTimeout(resolve, 2000));

  await Promise.all([prashantPromise, nikitaPromise]);
  const teamTime = Date.now() - start;

  await mayurPromise;
  const mayurTime = Date.now() - start;

  const winner = (teamTime < mayurTime) ? "Team" : "Mayur";
  console.log(`Winner is: ${winner}`);
}
```

(a) Mayur

(b) Team

(c) Undefined

(d) Error due to async logic

[Source ans: a]

2025 Oct26 Quiz-1

62. [Source Q4; MCQ]

Consider the following JavaScript code:

```
let employees = [
  { name: "Rahul", age: 28 },
  { name: "Priya", age: 24 },
  { name: "Amit", age: 32 }
];
employees.sort((a, b) => a.age - b.age);
console.log(employees.map(e => e.name).join(" - "));
```

What will be printed on the console?

(a) Rahul - Priya - Amit

(b) Priya - Rahul - Amit

(c) Amit - Rahul - Priya

(d) Priya - Amit - Rahul

[Source ans: b]

63. [Source Q5; MCQ]

Consider the following JavaScript code snippet.

```
const users = [
  { name: "Amit", age: 25 },
  { name: "Bhavna", age: 20 },
  { name: "Chirag", age: 30 },
];

const names = users
  .filter((u) => u.age >= 25)
  .map((u) => u.name.toUpperCase())
  .sort();

console.log(JSON.stringify(names));
```

What is logged on the console?

(a) ['Amit', 'Chirag']

(b) ['CHIRAG', 'AMIT']

(c) ['AMIT', 'CHIRAG']

(d) ['AMIT', 'Bhavna', 'CHIRAG']

[Source ans: c]

64. [Source Q6; MCQ]

Consider the following JavaScript code snippet.

```
let subject = "Dog";

function camera() {
  let subject = "Deer";
  function click() {
    console.log(`clicked picture of ${subject}`);
  }
  subject = "Cat";
  return click;
}

let cameraPicture = camera();
cameraPicture();
```

What is logged to the console?

(a) clicked picture of Dog

(b) clicked picture of Deer

(c) clicked picture of Cat

(d) clicked picture of undefined

[Source ans: c]

65. [Source Q12; MCQ]

Consider the following JavaScript code snippet:

```
const products = [
   { name: 'Laptop', price: 1200, category: 'Electronics', inStock: true },
   { name: 'Book', price: 25, category: 'Education', inStock: false },
   { name: 'Phone', price: 800, category: 'Electronics', inStock: true },
   { name: 'Pen', price: 5, category: 'Stationery', inStock: true },
   { name: 'Tablet', price: 600, category: 'Electronics', inStock: false }
];

const result = products
    .filter(item => item.category === 'Electronics' && item.inStock)
    .map(item => ({ ...item, discountedPrice: item.price * 0.9 }))
    .filter(item => item.discountedPrice > 500);

console.log(result.length);
console.log(result[0].name);
```

What will be the output of the above code on the browser's console?

(a)

```
3
Laptop
```

(b)

```
2
Laptop
```

(c)

```
1
Phone
```

(d)

```
2
Phone
```

[Source ans: b]

66. [Source Q13; MCQ]

Consider the below JavaScript program:

```
const globalVar = 50;

const object1 = {
    globalVar: 100,
    method1: function() {
        console.log(globalVar, this.globalVar);
        return () => {
            console.log(globalVar, this.globalVar);
        }
    }
}

const object2 = {
    globalVar: 200,
    method2: () => {
        console.log(globalVar, this.globalVar);
        return function() {
            console.log(globalVar, this.globalVar);
        }
    }
}

const fn1 = object1.method1();
fn1();
const fn2 = object2.method2();
fn2();
```

What will be the output of the above program on the browser's console, if executed?

(a)

```
50 100
50 100
50 undefined
50 undefined
```

(b)

```
50 100
50 100
50 undefined
50 200
```

(c)

```
100 50
100 50
200 50
200 50
```

(d)

```
50 100
50 100
50 undefined
50 this
```

[Source ans: a]

67. [Source Q14; MSQ]

Consider the following JavaScript code:

```
class Region {
  constructor(region) {
    this.region = region;
  }
  describe() {
    return `${this.region} is a region.`;
  }
}

class Country extends Region {
  constructor(region, name) {
    super(region);
    this.name = name;
  }
  describe() {
    return `${this.name} is in ${this.region}.`;
  }
}

let r = new Region("Asia");
let c = new Country("Europe", "Germany");
let f = c.describe;
```

Which of the following statements are **true**?

(a) Calling `r.describe()` prints **"Asia is a region."**

(b) Calling `c.describe()` prints **"Germany is in Europe."**

(c) Calling `f()` directly will still print **"Germany is in Europe."**

(d) The `Country` class overrides both the **constructor** and the **describe()** method from `Region`.

(e) The call to `super(region)` inside `Country`'s constructor is **not required** to initialize `this.region`.

[Source ans: a, b, d]

68. [Source Q15; MSQ]

Consider the following JavaScript function definitions. Which of the following will correctly return the product of three numbers and handle the case where any parameter is undefined by treating it as 1?

(a)

```
let multiply = (a, b, c) => {
    a = a || 1;
    b = b || 1;
    c = c || 1;
    return a * b * c;
}
```

(b)

```
function multiply(a = 1, b = 1, c = 1) {
    return a * b * c;
}
```

(c)

```
let multiply = (a, b, c) => {
    return (a ?? 1) * (b ?? 1) * (c ?? 1);
}
```

(d)

```
const multiply = function(a, b, c) {
    if(a === undefined) a = 1;
    if(b === undefined) b = 1;
    if(c === undefined) c = 1;
    a * b * c;
}
```

[Source ans: b, c]

2025 Dec18 ET

69. [Source Q1; MCQ]

Consider the following JavaScript code snippet:

```
const calculator = {
    multiplier: 5,
    calculate: function(a, b) {
        console.log(`Result: ${(a + b) * this.multiplier}`);
        return (a + b) * this.multiplier;
    }
};

const advancedCalc = {
    multiplier: 10,
    bonus: 20
};

const operation1 = calculator.calculate.call(advancedCalc, 3, 5);
const operation2 = calculator.calculate.bind(advancedCalc, 4);
operation2(8);
console.log(operation1);
console.log(typeof operation2);
```

What will be the output of the above program?

(a)

```
Result: 40
Result: 60
40
object
```

(b)

```
Result: 80
Result: 120
80
object
```

(c)

```
Result: 40
Result: 60
40
function
```

(d)

```
Result: 80
Result: 120
80
function
```

[Source ans: d]

70. [Source Q3; MCQ]

Consider the following JavaScript code snippet.

```
Promise.resolve(1)
  .then(v => { throw v + 1 })
  .catch(e => e * 2)
  .then(console.log)
```

What will be printed on the console?

(a) 2

(b) 4

(c) Undefined

(d) Error

[Source ans: b]

71. [Source Q13; MCQ]

Consider the following JavaScript code.

```
const p = Promise.resolve(1)
  .then(v => { console.log("A", v); return v + 1; })
  .then(v => { console.log("B", v); throw "err"; })
  .catch(e => { console.log("C", e); return 10; })
  .then(v => console.log("D", v));

console.log("END");
```

What will be the output?

(a)

```
END
A 1
B 2
C err
D 10
```

(b)

```
A 1
B 2
END
C err
D 10
```

(c)

```
END
A 1
C err
D 10
```

(d)

```
END
A 1
B 2
C err
```

[Source ans: a]

72. [Source Q14; MCQ]

Consider the following JavaScript snippet.

```
let arr = [1, 2, 3];
let result = arr.map(x => x * 2).filter(x => x > 4);
console.log(result);
```

What will be printed on the browser's console?

(a) [2, 4, 6]

(b) [6]

(c) [4, 6]

(d) [1, 2, 3]

[Source ans: b]

73. [Source Q15; MCQ]

Consider the following JavaScript snippet:

```
const obj = {
  x: 50,
  getX() { return this.x; }
};
const fn = obj.getX.bind({ x: 7 });
console.log(fn());
```

What will be printed on the browsers' console?

(a) 50

(b) undefined

(c) Error

(d) 7

[Source ans: d]

74. [Source Q19; MCQ]

A Promise chain is created as follows. The first Promise resolves with a string, which is then processed through multiple .then() handlers. What will be logged to the console?

```
const processString = (str) => {
   return new Promise((resolve) => {
       setTimeout(() => resolve(str.toUpperCase()), 100);
   });
};

Promise.resolve('hello')
   .then(processString)
   .then(result => result + ' WORLD')
   .then(console.log)
   .catch(err => console.log('Error:', err));
```

(a) hello

(b) hello WORLD

(c) HELLO WORLD

(d) An error will be thrown

[Source ans: c]

75. [Source Q21; MSQ]

Which of the following statements about the ”this” keyword in JavaScript are correct?

(a) In a regular function, ”this” is always bound to the global object (window in browsers)

(b) In an arrow function, ”this” is inherited from the enclosing lexical scope

(c) The .call() method can be used to explicitly set the context of ”this” in a function

(d) In a method called on an object, ”this” always refers to that object regardless of how the function is defined

[Source ans: b, c]

76. [Source Q24; MCQ]

A developer creates a Vue component with prototype chain inspection. The component uses class-based inheritance with lifecycle hooks. Consider the following scenario. Code:

```
class BaseComponent {
    constructor(name) {
        this.name = name;
    }
}

class AdvancedComponent extends BaseComponent {
    constructor(name, role) {
        super(name);
        this.role = role;
    }
}

const comp = new AdvancedComponent('Alice', 'admin');
console.log(comp.__proto__ === AdvancedComponent.prototype);
console.log(comp.__proto__.__proto__ === BaseComponent.prototype);
console.log(comp instanceof BaseComponent);
console.log(comp instanceof AdvancedComponent);
```

What will be the output on the browser’s console?

(a) true true true true

(b) true false true true

(c) false true true false

(d) true true false false

[Source ans: a]

2026 Mar15 Quiz-1

77. [Source Q2; MSQ]

Which of the following statements about JavaScript functions is/are true?

(a) Every function in JavaScript is an object.

(b) Arrow functions have their own "this" binding

(c) Function declarations are hoisted.

(d) Function expressions assigned to variables are hoisted with their function value.

[Source ans: a, c]

78. [Source Q3; MCQ]

Consider the following JavaScript code:

```
var a = 3;
const obj1 = {
  a: 20,
  show: function () {
    console.log(this.a);
  }
};
const obj2 = { a: 10 };

obj1.show.apply(obj1);
```

What will be printed on the console?

(a) 20

(b) 10

(c) undefined

(d) 3

[Source ans: a]

79. [Source Q7; MCQ]

What will be the output of the following program ?

```
const course = {
  courseName: 'Modern Application Development 2' ,
  courseCode: 'mad2' ,
}
const student = {
  __proto__: course,
  studentName: 'Rakesh' ,
  studentCity: 'Delhi' ,
}
const { courseName } = student
console.log(courseName)
```

(a) Undefined

(b) Modern Application Development 2

(c) Will throw syntax error

(d) None of these

[Source ans: b]

80. [Source Q11; MCQ]

Consider the below Javascript function:

```
function outer() {
    let count = 0;
    return function inner() {
        return count++;
    };
}

const c1 = outer();
const c2 = outer();

console.log(c1())
console.log(c1())
console.log(c2())
```

What will be the output of the above program ?

(a) 1 2 1

(b) 0 0 0

(c) 1 2 3

(d) 0 1 0

[Source ans: d]

81. [Source Q13; MCQ]

Consider the following JavaScript code:

```
function createCounter(start) {
  let count = start;

  return function (step) {
    if (typeof step === "number") {
      count += step;
      return count;
    }

    return (function () {
      let temp = count;

      return function (reset = false) {
        if (reset) {
          count = start;
        } else {
          count++;
        }
        return temp + count;
      };
    })();
  };
}

const counter = createCounter(3);

let a = counter(2);
let d = counter()(true);

a + d;
```

What would be the end result at the end of execution of the code?

(a) 5

(b) 13

(c) 8

(d) 11

[Source ans: b]

82. [Source Q14; MSQ]

Consider the following JavaScript code:

```
class Device {
  constructor(type) {
    this.type = type;
  }

  info() {
    return `${this.type} device`;
  }
}

class Mobile extends Device {
  constructor(type, brand) {
    super(type);
    this.brand = brand;
  }

  info(brand = "type") {
    return `${this.brand} is a ${this.type} device`;
  }
}

let d = new Device("Electronic");
let m = new Mobile("Electronic", "Samsung");

let x = m.info;
```

Which of the following statements are TRUE?

(a) `d.info()` returns ”Electronic device”

(b) `m.info()` returns ”Samsung is a Electronic device”

(c) Calling `x()` throws a runtime error

(d) `d.constructor()` returns ”Electronic device”

[Source ans: a, b, c]

2026 Apr06 Quiz-2

83. [Source Q4; MCQ]

Consider the following JavaScript code:

```
function task(value) {
  return new Promise((resolve, reject) => {
    if (value > 5) {
      resolve(value + 2);
    } else {
      reject(value - 2);
    }
  });
}

async function execute() {
  try {
    const a = await task(7);
    const b = await task(a - 4);
    return a + b;
  } catch (err) {
    return err;
  }
}

execute().then(result => console.log(result));
```

What will be printed to the console?

(a) 12

(b) 10

(c) 5

(d) 3

[Source ans: d]

84. [Source Q10; MCQ]

Consider the following code snippet running in a browser.

```
function inventory() {
  const weapon = { name: "Sword", stats: { damage: 10 } };

  const alias1  = weapon;
  const alias2  = { ...weapon };
  const alias3  = { name: weapon.name, stats: { ...weapon.stats } };

  alias1.name = "Shield";
  alias2.stats.damage = 99;
  alias3.stats.damage = 42;

  console.log(weapon.name);
  console.log(weapon.stats.damage);
  console.log(alias2.stats.damage);
  console.log(alias3.stats.damage);
}

inventory();
```

What will be the output on the console?

(a) Shield 99 99 42

(b) Shield 10 99 42

(c) Sword 99 99 42

(d) Shield 99 42 42

[Source ans: a]

85. [Source Q11; MCQ]

Consider the following code snippet running on a browser. What is the output on the console?

```
const obj = {
  val: 42,
  outer: function () {
    const inner = () => {
      console.log(this.val);
    };
    return inner;
  },
  alt: function () {
    console.log(this.val);
  }
};

const fn1 = obj.outer();
const fn2 = obj.outer;
const fn3 = obj.alt;

fn1();       // #1
fn2()();     // #2
fn3();       // #3
```

(a) 42 42 42

(b) 42 undefined undefined

(c) 42 undefined 42

(d) undefined undefined undefined

[Source ans: b]

2026 May06 ET

86. [Source Q1; MCQ]

Which of the following statements is true regarding the ”this” keyword in JavaScript?

(a) ”this” in an arrow function refers to the global object.

(b) ”this” inside a regular function refers to the object that owns the function.

(c) Arrow functions do not have their own ”this” and inherit it from the surrounding scope.

(d) ”this” always points to the window object in browser environments.

[Source ans: c]

87. [Source Q2; MCQ]

Consider the below JavaScript program.

```
let arr = [1, 2, 3];
let sum = arr.reduce((acc, val, idx, array) => acc + val, 0);
console.log(sum);
```

What will be the output of the above program?

(a) 0

(b) 3

(c) 6

(d) 9

(e) 12

[Source ans: c]

88. [Source Q5; MCQ]

You are building a cricket analytics app. The file battingStats.js contains the following exports:

```
export const strikeRate = (runs, balls) => {
  return (runs / balls) * 100;
};

export const playerName = "Virat Kohli";
```

In another file app.js, which of the following import statements will correctly import both strikeRate and playerName ? (Assume both files are in the same directory)

(a)

```
import strikeRate, playerName from "./battingStats.js";
```

(b)

```
import { strikeRate, playerName } from "./battingStats.js";
```

(c)

```
import * as stats from "./battingStats.js";
```

(d)

```
require("./battingStats.js");
```

[Source ans: b]

89. [Source Q7; MCQ]

Consider the below JavaScript program.

```
const promise = new Promise((resolve, reject) => {
 resolve('Success!');
 reject('Error!');
});
promise.then(console.log).catch(console.log);
```

What will be the output of the above program?

(a) Success!

(b) Error!

(c) Success! Error!

(d) Error! Success!

[Source ans: a]

90. [Source Q11; MCQ]

Consider the following JavaScript program:

```
class Animal {
    constructor(type) {
        this.type = type;
    }
    speak() {
        console.log(`${this.type} makes a sound`);
    }
    getType() {
        return `This is a ${this.type}`;
    }
}

class Dog extends Animal {
    constructor(type, breed) {
        super(type);
        this.breed = breed;
    }
    speak() {
        console.log(`${this.type} ${this.breed} barks`);
    }
}

const myDog = new Dog('Dog', 'Labrador');
myDog.speak();
console.log(myDog.getType());
console.log(myDog instanceof Dog);
console.log(myDog instanceof Animal);
console.log(Dog.prototype.__proto__ === Animal.prototype);
```

What will be the output of the above program on a browser's console?

(a)

```
Dog Labrador barks
This is a Dog
true
true
true
```

(b)

```
Dog makes a sound
This is a Dog Labrador
true
false
true
```

(c)

```
Dog Labrador barks
This is a Dog Labrador
true
true
false
```

(d)

```
Dog Labrador barks
This is a Dog
false
true
true
```

[Source ans: a]

91. [Source Q12; MCQ]

Consider the following JavaScript program:

```
const person = {
   firstName: 'John',
   lastName: 'Doe',
   age: 30,
   city: 'New York'
};

const { firstName, city, ...rest } = person;
const newPerson = { firstName, location: city, ...rest };

console.log(firstName);
console.log(rest);
console.log(newPerson);
```

What will be the output of the above program?

(a)

```
John
{ city: 'New York', lastName: 'Doe', age: 30 }
{ firstName: 'John', location: 'New York', city: 'New York', lastName:
'Doe', age: 30 }
```

(b)

```
John
{ lastName: 'Doe', age: 30 }
{ firstName: 'John', location: 'New York', lastName: 'Doe', age: 30 }
```

(c)

```
John
{ lastName: 'Doe', age: 30 }
{ firstName: 'John', location: 'New York', city: 'New York', lastName:
'Doe', age: 30 }
```

(d)

```
undefined
{ firstName: 'John', city: 'New York', lastName: 'Doe', age: 30 }
{ firstName: 'John', location: 'New York' }
```

[Source ans: b]

92. [Source Q14; MCQ]

Consider the following JavaScript program:

```
const CAPACITY = 75;
const TOTAL_PEOPLE = 213;

function allocatePeople() {
  return new Promise((resolve, reject) => {
    let slots = [0, 0, 0];
    let remaining = TOTAL_PEOPLE;

    for (let i = 0; i < slots.length; i++) {
      if (remaining >= CAPACITY) {
        slots[i] = CAPACITY;
        remaining -= CAPACITY;
      } else {
        slots[i] = remaining;
        remaining = 0;
      }
    }

    reject(slots);
  });
}

async function startAllocation() {
  try {
    const result = await allocatePeople();
    console.log("Passed " + JSON.stringify(result));
  } catch (error) {
    console.log("Failed " + JSON.stringify(error));
  }
}

startAllocation();
```

What will be printed in the console when the above code is executed?

(a) Passed [75,75,63]

(b) Failed [0,0,0]

(c) Failed [75,75,63]

(d) Passed [0,0,0]

[Source ans: c]

93. [Source Q20; MCQ]

Consider the following code snippet running in a browser. What will be the output on the console?

```
function inventory() {
 const item = { name: "Polycab", stats: { power: 20 } };

 const ref1  = item;
 const ref2  = { ...item };
 const ref3  = { name: item.name, stats: { ...item.stats } };

 ref1.name = "Ambrane";
 ref2.stats.power = 50;
 ref3.stats.power = 30;

 console.log(item.name);
 console.log(item.stats.power);
 console.log(ref2.stats.power);
 console.log(ref3.stats.power);
}

inventory();
```

(a) Ambrane 20 50 30

(b) Polycab 50 50 30

(c) Ambrane 50 50 30

(d) Ambrane 50 30 30

[Source ans: c]

94. [Source Q21; MCQ]

Consider the following code snippet running on a browser. What is the output on the console?

```
const obj = {
 value: 100,
 outerFunc: function () {
   const innerFunc = () => {
     console.log(this.value);
   };
   return innerFunc;
 },
 otherFunc: function () {
   console.log(this.value);
 }
};

const func1 = obj.outerFunc();
const func2 = obj.outerFunc;
const func3 = obj.otherFunc;

func1();        // #1
func2()();      // #2
func3();        // #3
```

(a) 100 100 100

(b) 100 undefined 100

(c) undefined undefined undefined

(d) 100 undefined undefined

[Source ans: d]

2026 Jul16 Quiz-1

95. [Source Q3; MSQ]

Which of the following cases is a function being used as a callback?

(a)

```
items.forEach(item => console.log(item))
```

(b)

```
setTimeout(showMessage, 1000)
```

(c)

```
const result = showMessage()
```

(d)

```
button.addEventListener("click", handleClick)
```

[Source ans: a, b, d]

96. [Source Q4; MCQ]

What is the correct output order for the following code?

```
console.log("1")

setTimeout(() => {
  console.log("2")
  setTimeout(() => console.log("3"), 0)
}, 0)

console.log("4")
```

(a) 1 2 3 4

(b) 1 4 2 3

(c) 2 1 4 3

(d) 4 1 2 3

[Source ans: b]

97. [Source Q7; MCQ]

Consider the following JavaScript code snippet.

```
const obj1 = {
   prop1: "Germany",
   prop2: "Spain",
   prop3: 50,
   method1: function(){
       console.log(`The European countries are ${this.prop2} and ${this.prop1}.`)
   },
   method2: function(param){
       let value = param + 2*this.prop3;
       console.log(`The final numerical value is: ${value}.`)
   }
}

const obj2 = {
   prop1: "England",
   prop2: "Italy",
   prop3: "25"
}

const func1 = obj1.method1;
const func2 = obj1.method2;

func1.apply(obj1);
func2.bind(obj2)(15);
```

What will be the output of the above code on the browsers' console?

(a) The European countries are Spain and Germany. The final numerical value is: 115.

(b) The European countries are Italy and England. The final numerical value is: 65.

(c) The European countries are Spain and Germany. The final numerical value is: 65.

(d) The European countries are Italy and England. The final numerical value is: 115.

[Source ans: c]

98. [Source Q8; MCQ]

Consider the following JavaScript code snippet.

```
const obj1 = {
   prop1: "Aluminium",
   prop2: "Gold",
   prop3: "Silver"
}

const obj2 = {
   propA: "Argon",
   propB: "Neon"
}

const {prop2: element1, propA: element2} = {...obj2, ...obj1};

console.log(`The selected elements are: ${element1} and ${element2}`);
```

What will be the output of the above code on the browsers' console?

(a) The selected elements are: Gold and Argon

(b) The selected elements are: Aluminium and Silver

(c) The selected elements are: Argon and Neon

(d) The selected elements are: Gold and Neon

(e) The selected elements are: Argon and Gold

[Source ans: a]

99. [Source Q12; MSQ]

Consider the following JavaScript code snippet.

```
class MathUtils {
    static add(a, b) {
        return a + b;
    }
    multiply(a, b) {
        return a * b;
    }
}

const utils = new MathUtils();
```

Which of the following statements is/are correct?

(a) `MathUtils.add(5, 3)` will work correctly and return 8

(b) `utils.add(5, 3)` will work correctly and return 8

(c) `utils.multiply(5, 3)` will work correctly and return 15

(d) `MathUtils.multiply(5, 3)` will work correctly and return 15

[Source ans: a, c]

100. [Source Q13; MCQ]

Consider the following JavaScript code snippet.

```
var var1 = 25;
var var2 = 35;

const obj = {
   var1: 45,
   var2: 55,
   func1: function(){
       console.log(`You are accessing: ${var1}`)
   },
   func2: () => {
       console.log(`You are accessing: ${this.var2}`)
   },
}

obj.func1()
obj.func2()
```

What will be the output of the above code on the browsers' console?

(a) You are accessing: 45 You are accessing: 55

(b) You are accessing: 25 You are accessing: 35

(c) You are accessing: 25 You are accessing: 55

(d) You are accessing: 45 You are accessing: 35

[Source ans: b]

101. [Source Q15; MCQ]

Consider the following JavaScript code snippet.

```
let myArr = ["January", "March", "April", "August", "September", "October"]

let modArr = myArr.map((m) => m.slice(2)).
                   filter((m) => m.length < 5).
                   sort();
console.log(modArr[1]);
```

What will be the output of the above code on the browsers' console?

(a) ['gust', 'rch', 'ril']

(b) ['rch', 'ril', 'gust']

(c) ril

(d) rch

[Source ans: d]

102. [Source Q16; MCQ]

Consider the following JavaScript code snippet.

```
function first() {
    console.log("First start");
    setTimeout(function () {
        console.log("First timeout");
    }, 0);
    console.log("First end");
}

function second() {
    console.log("Second start");
    setTimeout(function () {
        console.log("Second timeout");
    }, 0);
}

console.log("Program start");
first();
second();
console.log("Program end");
```

What will be the output of the above code on the browsers' console?

(a) Program start First start First timeout First end Second start Second timeout Program end

(b) Program start First start First end First timeout Second start Second timeout Program end

(c) Program start First timeout First start First end Second timeout Second start Program end

(d) Program start First start First end Second start Program end First timeout Second timeout

[Source ans: d]

2026 Aug14 Quiz-2

103. [Source Q4; MCQ]

What is the correct output order for the following code?

```
console.log("A")
setTimeout(() => console.log("B"), 0)
Promise.resolve().then(() => console.log("C"))
console.log("D")
```

(a) A B C D

(b) C A D B

(c) A D C B

(d) A D B C

[Source ans: c]

104. [Source Q5; MCQ]

Consider the following code:

```
const settings = { theme: "light", layout: { sidebar: true } }
const copy = { ...settings }
copy.layout.sidebar = false
```

What happens to

`settings.layout.sidebar`

?

(a) It remains `true` because spread syntax always creates a deep copy.

(b) It becomes `false` because the nested object is still shared in a shallow copy.

(c) It becomes `undefined` because `layout` is removed from the original object.

(d) The code throws an error because nested objects cannot be copied with spread syntax.

[Source ans: b]

105. [Source Q6; MCQ]

Consider the following JavaScript code snippet?

```
async function fetchData() {
 console.log("1");
  const result = await new Promise((resolve) => {
   setTimeout(() => resolve("done"), 1000);
   console.log("2");
 });

 console.log("3:", result);
}

fetchData();
console.log("4");
```

What will be the output of above code on the browser's console?

(a) $1 \rightarrow 2 \rightarrow 3$: done $\rightarrow 4$

(b) $1 \rightarrow 4 \rightarrow 2 \rightarrow 3$: done

(c) $1 \rightarrow 2 \rightarrow 4 \rightarrow 3$: done

(d) $1 \rightarrow 2 \rightarrow 4$ (3 never prints due to unresolved promise)

[Source ans: c]

2026 Sep13 ET

106. [Source Q12; MCQ]

What is logged to the console after running the following code?

```
const numbers = [10, 20, 30, 40, 50];
const [num1, , num2, ...rest] = numbers;
const [a = 100, b = 200] = [undefined, 0];

console.log(num1, num2, rest);
console.log(a, b);
```

(a) 10 30 [30, 40, 50] and 100 0

(b) 10 30 [40, 50] and 100 200

(c) 10 20 [30, 40, 50] and undefined 0

(d) 10 30 [40, 50] and 100 0

[Source ans: d]

107. [Source Q13; MCQ]

What is logged to the console after running the following code?

```
const user = {
  id: 1,
  profile: {
    name: "Riya",
    address: {
      city: "Pune",
      zip: "411001"
    }
  }
};

const {
  profile: {
    name,
    address: { city, state = "Maharashtra" }
  },
  role = "guest",
  profile
} = user;

console.log(name, city, state, role);
console.log(profile);
```

(a)

```
Riya Pune Maharashtra guest
{ name: "Riya", address: { city: "Pune", zip: "411001" } }
```

(b)

```
Riya Pune Maharashtra guest
undefined
```

(c)

```
undefined undefined Maharashtra guest
{ name: "Riya", address: {...} }
```

(d) This code throws a `SyntaxError` , because you cannot destructure profile into nested variables and also assign it to a plain variable profile in the same statement.

[Source ans: a]

108. [Source Q14; MCQ]

Consider the following JavaScript code:

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
Promise.resolve().then(() => {
  console.log("5");
  setTimeout(() => console.log("6"), 0);
});
```

What is the order of console output?

(a) 1, 4, 3, 5, 2, 6

(b) 1, 4, 3, 5, 6, 2

(c) 1, 3, 4, 5, 2, 6

(d) 1, 4, 2, 3, 5, 6

[Source ans: a]

109. [Source Q18; MSQ]

Given the following code, select all the statements that are TRUE about its execution if the code is run in a browser environment.

```
const calculator = {
  value: 10,
  add(a, b) {
    return this.value + a + b;
  }
};

const rawAdd = calculator.add;
const boundAdd = calculator.add.bind({ value: 100 }, 5);

console.log(rawAdd(1, 2));
console.log(boundAdd(10));
console.log(calculator.add.apply({ value: 50 }, [1, 2]));
```

(a) `rawAdd(1, 2)` evaluates to `NaN`

(b) `boundAdd(10)` evaluates to `115`

(c) `calculator.add.apply({ value: 50 }, [1, 2])` evaluates to `53`

(d)

`calculator.add.apply({ value: 50 }, [1, 2])`

throws a `TypeError`

[Source ans: a, b, c]

110. [Source Q20; MSQ]

Given the following code running in the browser, select all the statements that are `TRUE` about its output:

```
const value = "global";

const obj = {
  value: "objectValue",
  regularMethod: function () {
    console.log(this.value);
  },
  arrowMethod: () => {
    console.log(this.value);
  },
  nested: {
    value: "nestedValue",
    regularInNested: function () {
      console.log(this.value);
    },
    arrowInNested: () => {
      console.log(this.value);
    }
  },
  delayedRegular: function () {
    setTimeout(function () {
      console.log(this.value);
    }, 100);
  },
  delayedArrow: function () {
    setTimeout(() => {
      console.log(this.value);
    }, 100);
  }
};
```

```
obj.regularMethod();
obj.arrowMethod();
obj.nested.regularInNested();
obj.nested.arrowInNested();
obj.delayedRegular();
obj.delayedArrow();
```

(a) `obj.regularMethod()` logs ”objectValue”

(b) `obj.arrowMethod()` logs ”objectValue”

(c) `obj.nested.regularInNested()` logs ”nestedValue”

(d) `obj.nested.arrowInNested()` logs `undefined`

(e) `obj.delayedArrow()` logs ”objectValue”

(f) `obj.delayedRegular()` logs ”objectValue”

[Source ans: a, c, d, e]

Shared context (source Q22)

Consider the following javascript code:

```
const user = {
  name: "Rahul",
  skills: ["Python", "SQL"]
}
```

Note: Each sub question runs on its own, starting from the original ’user’ object. Based on the above data, answer the given subquestions.

111. [Source Q23; MCQ]

What will be logged in the console ?

```
const cloned = { ...user }

cloned.name = "Amit"
cloned.skills.push("JavaScript")

console.log(user.name)
console.log(user.skills.length)
```

(a) Rahul 2

(b) Amit 3

(c) Rahul 3

(d) Amit 2

[Source ans: c]

112. [Source Q24; MCQ]

What will be logged in the console ?

```
const cloned = {
  ...user,
  skills: [...user.skills]
}

cloned.skills.push("JavaScript")

console.log(user.skills.length)
console.log(cloned.skills.length)
```

(a) 2 3

(b) 3 3

(c) 2 2

(d) 3 2

[Source ans: a]

Shared context (source Q25)

Consider the following JavaScript code:

```
function fetchData() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("Data Received")
    }, 1000)
  })
}

async function display() {
  console.log("Start")

  const result = await fetchData()

  console.log(result)
  console.log("End")
}

display()
```

Based on the above data, answer the given subquestions.

113. [Source Q26; MCQ]

What will be the output order?

(a) Start End Data Received

(b) Data Received Start End

(c) Start Data Received End

(d) End Start Data Received

[Source ans: c]

114. [Source Q27; MSQ]

Which of the following statements are correct?

(a) `await` pauses the execution of the `display()` function until the Promise resolves.

(b) `fetchData()` immediately returns the resolved string "Data Received".

(c) `async` functions cannot return a `Promise` .

(d) Removing `await` would cause `result` to store a `Promise` object instead of the resolved value.

[Source ans: a, d]